



Software Engineering Department
Braude College of Engineering
Final Project – Phase B (61999)

Horizon Forecast

Satellite Cloud IR Nowcasting Transformer Model

CODE: 26-1-R-1

By:

Gilad Boudman
Or Mordechay Hod

Advisors:

Mrs. Elena Kramer
Dr. Dan Lemberg

Link to GitHub:

<https://github.com/Horizon-Forecast/Satellite-Cloud-IR-Nowcasting-Transformer-Model>

Contents

Abstract	2
1 Background and Problem Review	3
2 What We Did	4
2.1 General Description	4
2.2 The Solution: Driver-First Cascade Architecture	5
2.3 Development and Research Process	7
2.4 Challenges and Solutions	9
2.5 Results and Conclusions	10
2.6 Lessons Learned	12
2.7 Meeting the Project Metrics	13
2.8 Future Work	14
References	15
Appendices	16
A User Guide	16
A.1 What the System Does	16
A.2 Use Scenario 1: Reproducing the Results	16
A.2.1 Prerequisites	16
A.2.2 Step 1: Run the Held-Out Evaluation	16
A.2.3 Interpreting the Outputs	17
A.3 Use Scenario 2: The Offline Demo	17
A.4 Scope and Limitations	18
B Maintenance and Developer Guide	19
B.1 Operating Environment	19
B.2 Repository Structure	19
B.3 Dataset Acquisition	20
B.4 Data Pipeline	22
B.5 Training	22
B.6 Evaluation	23
B.7 Extending the System	23
B.8 Rebuilding the Demo	24
B.9 Troubleshooting	24
B.10 Conventions	24

Abstract

Climate change has intensified the frequency and severity of extreme weather events, creating an urgent need for precise, real-time nowcasting systems. Traditional Numerical Weather Prediction (NWP) models suffer from significant latency, while standard end-to-end Deep Learning approaches often fail to capture the physical consistency between atmospheric drivers (Wind, Temperature) and surface precipitation. A further challenge is the sparsity of ground-truth rain data, which is available only at scattered station points and limits the achievable skill of purely visual forecasting models.

This project presents Horizon Forecast, a Driver-First Cascade nowcasting model based on SimVPv2 and SaTformer. Rather than mapping satellite frames directly to rain, the system first predicts the underlying thermodynamic drivers (Wind and Temperature), then uses these predicted drivers to condition a second stage that generates the future cloud structure and rain intensity. This forces the model to resolve the physical state of the atmosphere before committing to a precipitation forecast.

The model was trained and evaluated on satellite imagery, reanalysis data, topography, and ground station observations over Israel and the Eastern Mediterranean, using a strictly chronological train/validation/test split to ensure realistic generalization to unseen weather. Compared against a matched end-to-end single-stage baseline, the Driver-First Cascade substantially improved rain forecasting skill, while cloud-structure quality remained comparable between the two. This indicates that the cascade’s advantage is specific to precipitation forecasting: explicitly resolving the physical drivers before generation improves the accuracy of the rain forecast itself, rather than producing a broadly sharper visual output.

Keywords: Next-Frame Prediction, Driver-First Cascade, SimVPv2, Physics-Informed AI, Weather Nowcasting, Sparse Supervision.

1 Background and Problem Review

Climate change has intensified the frequency and severity of extreme weather events, increasing the demand for precise, real-time nowcasting systems capable of predicting rainfall and storm evolution typically on 0–4 hour horizons [5]. Traditional Numerical Weather Prediction (NWP) models solve the governing partial differential equations of the atmosphere directly, but this computational cost introduces latency that makes them unsuitable for rapidly evolving, localized hazards [1]. Deep Learning (DL) emerged as a faster, data-driven alternative: starting from the ConvLSTM [7], spatiotemporal predictive models evolved along two paths. Fully convolutional architectures such as SimVPv2 replace recurrence with a Gated Spatiotemporal Attention (gSTA) mechanism, achieving strong accuracy at low computational cost [8]. Transformer-based models such as SaTformer instead treat rainfall as a classification problem over intensity buckets, using full space-time self-attention and a class-weighted loss to better resolve rare, extreme events [3].

Both families, however, share two limitations that motivated this project. First, models trained with a regression loss such as MSE tend to average over possible futures, producing blurry predictions that under-represent sharp, high-intensity events [3]. Second, standard architectures are effectively physics-blind: they map satellite frames to future frames as a pure visual translation task, without an explicit representation of the atmospheric drivers – wind, temperature – that physically cause precipitation [6]. This is compounded by a data-availability problem specific to this domain (Israel): while satellite imagery gives dense spatial coverage, rain intensity is measured only at scattered IMS ground stations, and in the Eastern Mediterranean climate, rainfall itself is a rare event – our analysis of IMS records (2020–2025) found measurable rain in under 1.2% of timestamps in the north and under 0.25% in the arid south [4]. A model optimizing average pixel error can therefore reach low error by simply predicting “no rain” everywhere, while still failing at the task it was built for.

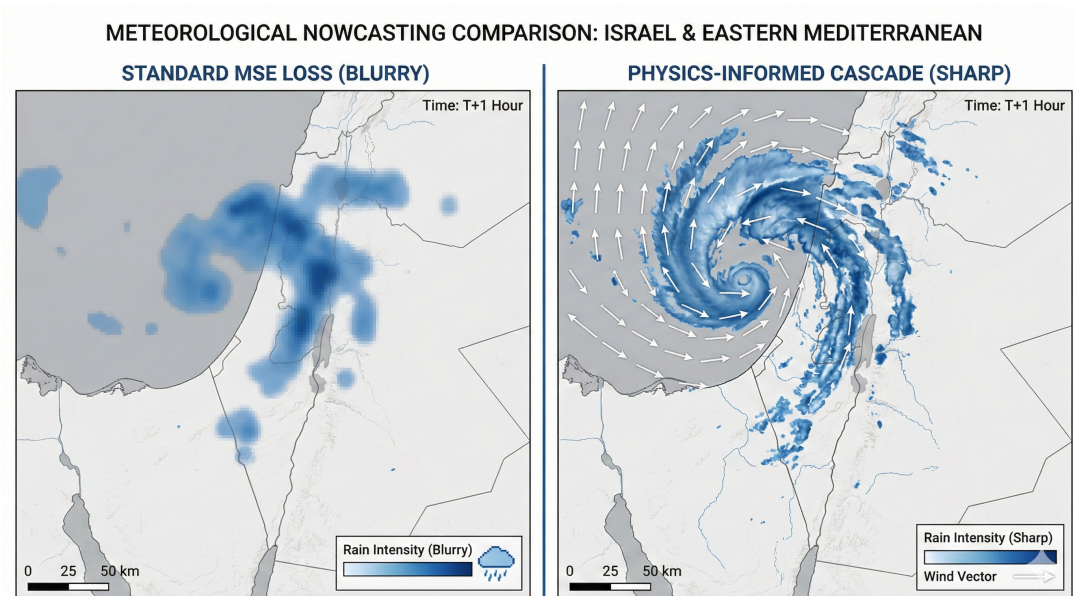


Figure 1: **Conceptual Illustration.** Standard MSE loss typically yields blurry predictions (Left), while a physics-informed architecture aims to achieve the sharp structural definition depicted on the Right. (Image generated using Google Nano Banana Pro).

This project’s central hypothesis is that these two problems are linked: a model forced to first resolve

why a storm is moving – its physical drivers – before generating where it goes should produce sharper, more physically consistent precipitation forecasts than a model that predicts rain directly from raw imagery. Horizon Forecast tests this hypothesis with a Driver-First Cascade architecture, combining the computational efficiency of SimVPv2 as a shared encoder with a two-stage decision process inspired by SaTformer’s event-focused loss design. The remainder of this book describes the resulting system, how it was built, and whether the driver-first hypothesis held under empirical evaluation.

2 What We Did

2.1 General Description

Horizon Forecast is a deep-learning nowcasting system built specifically for the Israel region. From the last hour of weather-satellite imagery (four frames at a 15-minute cadence) together with a static elevation map, it forecasts the near future of four fields over a fixed 256×256 grid covering Israel: surface wind, surface temperature, cloud structure, and rain intensity. Forecasts are produced autoregressively, one 15-minute step at a time. Training and evaluation focus on the short-range horizons (T+15 and T+30), while the same autoregressive loop can be continued to extend the forecast further.

The system’s distinguishing feature is its Driver-First Cascade design. Instead of mapping past satellite frames directly to future rain, the model first predicts the physical drivers of the weather (wind and temperature), and only then predicts the resulting clouds and rain conditioned on those drivers – “understand why the weather moves before predicting where it goes.” The primary goal of the project was to test whether this driver-first ordering measurably improves rain forecasting, evaluated through a controlled ablation against an otherwise identical end-to-end model. A secondary goal was practical: the trained model is compact (~ 30 M parameters) and runs on a consumer GPU, so a full multi-field forecast can be generated in well under the 15-minute data-refresh interval.

The system targets two kinds of users. The first is the research audience – students and researchers studying physics-informed forecasting architectures, for whom the built-in cascade/ablation switch and the evaluation pipeline are the main product. The second is the operational forecaster in Israel, who needs a fast, locally-tuned rain and cloud nowcast to complement Numerical Weather Prediction (NWP) guidance during the short-range window where NWP output is not yet available. For presentation, the project also includes a small standalone demo application that displays pre-computed forecasts for a few example cases across several time steps, so the model’s output can be shown and compared step by step without a GPU or a Python environment.

Data Sources

The system is trained on a custom, region-specific dataset fusing four sources over Israel and the Eastern Mediterranean (29–34°N, 34–36°E):

- **Satellite imagery (EUMETSAT SEVIRI):** IR $10.8\mu\text{m}$ (cloud-top height, convective structure, day and night) and WV $6.2\mu\text{m}$ (upper-tropospheric moisture / steering flow), from the Meteosat Second Generation Data Store [2].

- **Topography (NASA SRTM):** a static 256×256 digital elevation model, concatenated to every input frame, since Eastern Mediterranean rainfall is strongly modulated by terrain.
- **Reanalysis (ERA5):** dense 256×256 grids of Surface Wind Speed and Surface Temperature, used as the Stage 1 driver ground truth. Because ERA5 covers the full spatial domain, no masking is required for this target.
- **Ground stations (IMS):** sparse, point-based Precipitation Intensity (mm/hr) measurements, used as the Stage 2 rain ground truth. Because stations are scattered, a pre-computed Station Coordinate Mask restricts the rain loss to pixels with an active station, preventing the model from learning "zero rain" in unobserved regions.

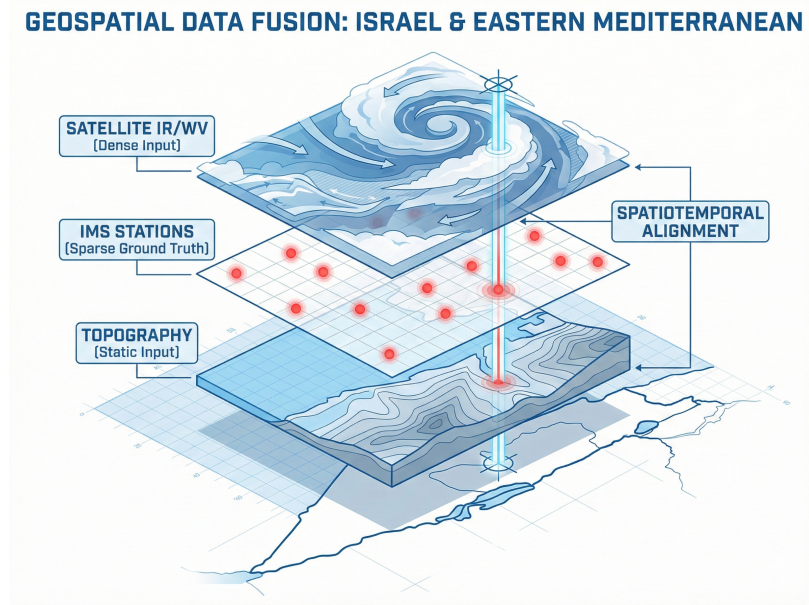


Figure 2: **Data Fusion Strategy.** The system aligns static topography, sparse point-based rain measurements, and dense satellite/reanalysis imagery onto a unified 256×256 grid over the Israeli domain. (Image generated using Google Nano Banana Pro).

The dataset spans five years (Jan 2020 – Dec 2025) and is split strictly chronologically to prevent temporal leakage: training on 2020–2023, validation on Jan–Jun 2024 for hyperparameter tuning, and a held-out test set from Jul 2024 – Dec 2025 (52,566 samples) used only for final benchmarking.

2.2 The Solution: Driver-First Cascade Architecture

The core of the system is a single neural network of roughly 30 million parameters, organized as one shared encoder followed by a two-stage cascade of prediction heads. The design deliberately separates the two questions a forecast must answer – why is the weather moving, and where will it go – into two ordered stages, so that the second stage is conditioned on the first.

Component Structure

At a package level the model is composed of three modules, executed in sequence for each 15-minute forecast step:

- **SimVPv2 Encoder (shared body).** Takes the stacked input tensor – four past frames of three channels each (IR, WV, DEM), flattened to 12 channels at 256×256 – and compresses it into a compact latent representation. A convolutional stem reduces the spatial resolution four-fold, and a stack of gated spatiotemporal attention (gSTA) blocks then captures motion at multiple scales, from the movement of individual clouds up to the broad wind currents that steer entire weather systems, at linear rather than quadratic cost.
- **Stage 1 — Physics Driver Head.** From the shared latent features, a lightweight convolutional decoder predicts the two dense driver fields – surface wind speed and surface temperature – back at full 256×256 resolution. This stage is supervised against dense ERA5 reanalysis.
- **Stage 2 — Manifestation Head.** The predicted driver fields are projected back into the latent space and fused with the encoder features (channel-wise concatenation followed by a blending convolution). A transformer attention stage then relates far-apart regions of the grid to one another, giving the model the wide-area view it needs to reason about whole storm systems rather than isolated pixels. The fused representation is then decoded into two outputs: the future cloud field (IR + WV) and the rain intensity, expressed as a 64-class probability distribution over intensity buckets. Rain class is recovered per pixel by taking the most probable bucket.

The cascade link between the two stages is the single most important design element. It is also made switchable: a one-flag ablation zeroes the driver fields before they reach Stage 2, leaving every other part of the network – weights, capacity, training data – identical. Because only the driver connection changes, comparing the two modes directly measures how much the driver-first ordering contributes.

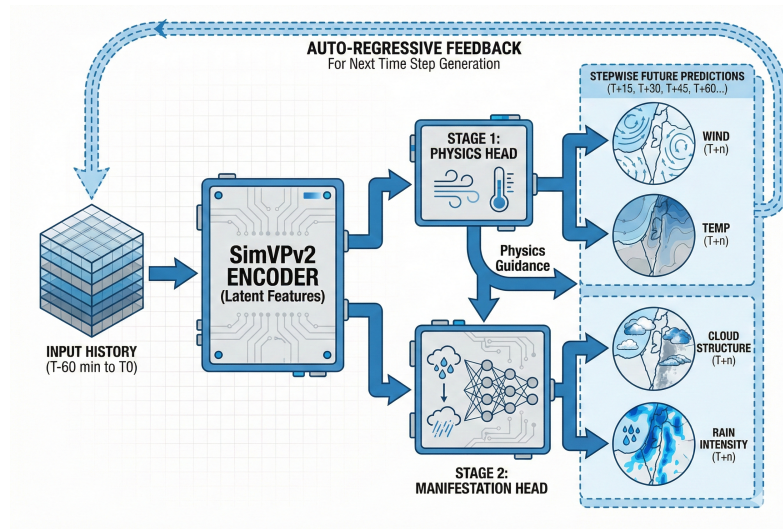


Figure 3: **System Architecture.** The shared SimVPv2 encoder feeds a two-stage cascade: Stage 1 predicts the thermodynamic drivers (wind, temperature), these are fused back with the latent features to condition Stage 2, which predicts cloud structure and rain intensity. The dashed feedback loop is the autoregressive mechanism used to extend the forecast step by step ($T+15$, $T+30$, ...). (Image generated using Google Nano Banana Pro).

Forecast Flow

A single forecast is produced by the following flow, repeated autoregressively to reach longer horizons:

1. **Ingest.** The four most recent satellite frames (IR, WV) are stacked with the static elevation map into one 12-channel input tensor.
2. **Encode.** The encoder compresses this tensor into the shared latent representation.
3. **Predict drivers (Stage 1).** The Physics Driver Head produces the wind and temperature fields for the next step.
4. **Fuse.** The driver fields are projected and concatenated with the latent features, forcing the next stage to condition on the physical state.
5. **Predict manifestation (Stage 2).** The Manifestation Head produces the next-step cloud field and the per-pixel rain-intensity class.
6. **Roll forward.** The predicted cloud field is combined with the elevation map to form a synthetic next frame, which slides into the input window, the flow repeats from step 2 to advance the forecast to T+30, T+45, and onward to T+120.

Compute Environments

The same model definition was used across two training environments, chosen mainly to control cost. Most development and iteration were done on a local NVIDIA RTX A5000 (24 GB), avoiding cloud charges while we tuned the architecture and training recipe. A paid cloud NVIDIA H100 was rented only for the final, full-dataset runs, where its much higher throughput cut training time from days to hours. These runs used mixed-precision compute and gradient checkpointing to fit the autoregressive rollout in memory. To further limit both training time and cloud cost, the rollout used during training and headline evaluation was capped at a T+30 horizon (two autoregressive steps) rather than the full two-hour rollout the architecture supports. Inference, by contrast, is deliberately lightweight: the trained weights load in half precision and run a full multi-field forecast on a consumer 8 GB GPU (an RTX 3070), so the model can be run on ordinary hardware once trained.

2.3 Development and Research Process

The project was carried out as a research investigation rather than a fixed feature build: the driver-first cascade was a hypothesis, and most of the engineering effort went into building the pipeline needed to test it fairly and into the sequence of experiments that refined it.

Data Pipeline

Before any model could be trained, the four raw data sources had to be fetched and aligned onto a single grid. We built a ten-stage preprocessing pipeline that: cleans and geolocates the IMS station records, resizes the NASA elevation map to the 256×256 grid, converts the raw SEVIRI satellite tiles (IR and WV)

into per-timestamp arrays, aligns the 10-minute station data and the ERA5 reanalysis to the 15-minute satellite timeline, builds the station-coordinate mask, constructs the chronological train/validation/test indices, and finally computes the normalization statistics and the per-class rain weights used by the loss. Each stage skips work that is already on disk and can resume after an interruption, so the multi-day preparation of a five-year dataset could be run incrementally without repeating completed work.

Experimental Iteration

The model was reached through a series of controlled experiments, each fixing a specific problem observed in the previous one:

- An early cascade trained with a single combined weight for the wind and temperature losses was numerically unstable and diverged. Splitting that into separate wind and temperature loss weights (keeping wind, the primary driver, stronger) stabilized Stage 1.
- Supervising the driver stage against dense ERA5 reanalysis, instead of only the sparse station points, cut the driver-field error by roughly thirty times – the single largest improvement, and the core Phase B change.
- To make the rain head actually use the driver signal rather than ignore it, we adopted a two-phase frozen training schedule: first the encoder and driver head are trained with the manifestation head frozen, then the encoder and driver head are frozen and the manifestation head is trained on top of the now-fixed physics features.
- The cascade was then compared against its own ablation (driver fields zeroed before Stage 2) to measure the contribution of the driver-first ordering in isolation.

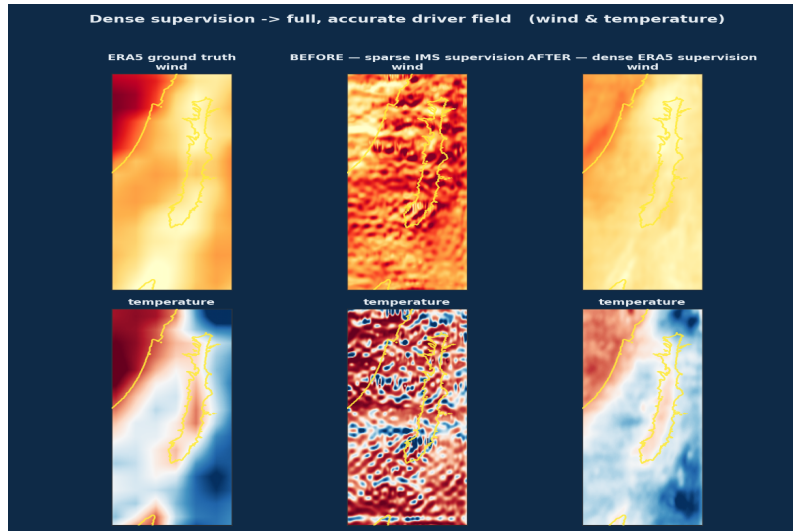


Figure 4: **Effect of dense driver supervision.** Predicted wind (top) and temperature (bottom) fields against the ERA5 ground truth (left). Supervising only the sparse IMS station points (middle) leaves a noisy, blocky field between stations, whereas dense ERA5 supervision (right) produces a smooth, physically plausible field that closely matches the ground truth. This change reduced the driver-field error by roughly thirty times.

A significant mid-project setback was the discovery that the two satellite channels (IR and WV) had been swapped in a subset of the source files. After correcting this, the affected models were retrained. Throughout, final checkpoints were selected by their rain skill score (CSI) rather than by validation loss, because the two did not move together: a model could keep driving its loss down while its actual ability to detect rain flattened out or even worsened.

Tools and Technologies

The system is built entirely in Python, organized into the following stack:

- **Model and training — PyTorch.** The network, the training loop, mixed-precision compute, and checkpointing are all implemented directly, giving full control over the cascade and the two-phase frozen schedule.
- **Data acquisition — EUMDAC.** The official EUMETSAT client is used to download the historical SEVIRI satellite archive.
- **Data processing — NumPy, Pandas, SciPy, and OpenCV.** Used to clean the station records, resize the elevation map, and align the multi-source data onto the common grid.
- **Evaluation and visualization — scikit-image and Matplotlib.** scikit-image provides the SSIM metric (the CSI and HSS skill scores are implemented directly), and Matplotlib renders the forecast figures.
- **Packaging — PyInstaller.** Bundles the demo viewer into a single standalone executable that runs without a Python install.
- **Version control — Git and GitHub.** Used for source control and collaboration between the two team members.

Usage of AI Tools

In accordance with academic integrity guidelines, we declare the following use of generative AI tools during Phase B:

- **Large Language Models.** Used in a supporting role during development – help with debugging code, discussing trade-offs when weighing design decisions, and language and formatting help while writing this book. Any AI-assisted code or text was reviewed and rewritten by us before use.
- **Image Generation (Google Nano Banana Pro).** Used to produce the conceptual illustration figures in this book. The result figures, by contrast, were plotted from the model’s actual outputs using our own code.

2.4 Challenges and Solutions

Building a rain nowcaster for this region surfaced several problems that were as much about the data as about the model. The most important ones, and how we addressed them, are described below.

Extreme Class Imbalance

Rain is a rare event in the Eastern Mediterranean: in our training data over 98% of station readings are “dry.” A model minimizing average error can therefore score very well by never predicting rain at all, which is exactly the failure mode we needed to avoid. We countered this on two fronts: the rain loss is a class-weighted cross-entropy that penalizes missing a rain event far more heavily than a false alarm, and we selected final models by their rain skill score (CSI) rather than by validation loss, since a model can keep improving its loss while its actual rain detection stalls.

Sparse Rain Ground Truth Caps Achievable Skill

Unlike the driver fields, which have dense ERA5 supervision at every pixel, rain is only observed at roughly a hundred scattered stations. This limits not just training but the measurable skill itself: a correct storm predicted a few kilometres from the nearest gauge is scored as both a miss and a false alarm, holding the pointwise CSI low regardless of how well the model actually places the rain.

Cross-Stage Gradient Interference

Training both cascade stages jointly caused the two objectives to fight: gradients from the rain/cloud head disturbed the encoder features the driver head depended on, and the rain head tended to ignore the driver signal entirely. We solved this with a two-phase frozen schedule – first train the encoder and driver head alone, then freeze them and train the manifestation head on top of the now-stable physics features – so each stage optimizes against a fixed target rather than a moving one.

Memory and Compute Limits

Every extra rollout step adds another forward pass that has to be held in memory during training, which quickly filled the GPU. Two techniques kept this in check: gradient checkpointing, which avoids storing every intermediate result, and mixed-precision compute, which halves the memory each value uses. On top of this the model was kept small ($\sim 30\text{M}$ parameters), so that once trained it runs on an ordinary 8 GB GPU rather than only on the high-memory hardware used for training.

A Silent Data Bug

Mid-project we discovered that the two satellite channels (IR and WV) had been swapped in a subset of the source files – a bug that quietly degraded results without ever crashing. Once found, it was corrected and the affected models were retrained. It was a reminder that in a data-driven system the most damaging errors are often in the data, not the code.

2.5 Results and Conclusions

All results below come from the strictly held-out test set (Jul 2024 – Dec 2025), which the model never saw during training or tuning. The two models compared are the Driver-First Cascade and the End-to-End Ablation, which uses the same architecture, training procedure, and datasets, but without providing the

predicted driver fields to Stage 2. Therefore, any difference between them can be attributed to the driver-first design. Rain is scored with the Critical Success Index (CSI, higher is better) at both a light-rain threshold and a heavy-rain threshold. Cloud quality is scored with SSIM, and we also report Probability of Detection (POD) and False Alarm Ratio (FAR).

Model	Rain CSI	Heavy-rain CSI	Cloud SSIM	POD	FAR
Driver-First Cascade	0.135	0.114	0.813	0.80	0.861
End-to-End Ablation	0.089	0.017	0.822	0.95	0.911

Table 1: **Cascade vs. Ablation on the held-out test set (T+15).** The driver-first ordering improves rain skill (CSI) substantially while leaving cloud quality (SSIM) essentially unchanged.

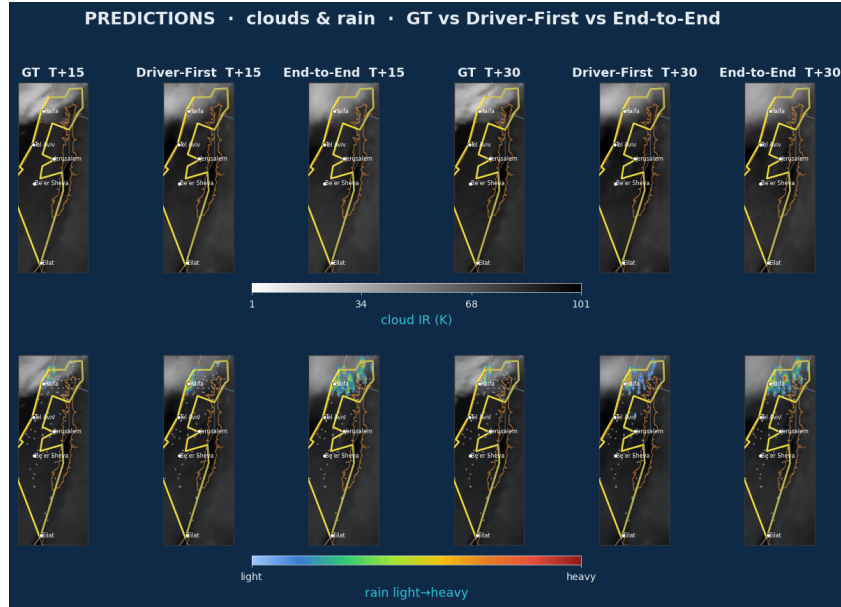


Figure 5: **Qualitative forecast for a representative storm case from the held-out test set.** Top row: cloud field (IR), bottom row: rain intensity. For each horizon (T+15, T+30) the panels show, left to right, the ground truth, the Driver-First Cascade, and the End-to-End Ablation.

The Driver-First Hypothesis Holds

The central result is that the cascade improves rain CSI by roughly **52%** over the identical end-to-end model (0.135 vs. 0.089). The gap is even larger for heavy rain, where the cascade reaches a CSI of 0.114 against the ablation’s 0.017 – the ordering helps most for the intense, high-impact events that matter most to forecast correctly. Notably, the cascade achieves this while predicting rain less often (lower POD) but far more precisely (lower FAR): the end-to-end model compensates for its weaker signal by over-predicting rain almost everywhere, whereas the driver-conditioned model is more selective and therefore more useful.

The Advantage Is Specific to Rain

Cloud-structure quality (SSIM) is essentially tied between the two models (~ 0.81 vs. ~ 0.82). This is an important finding in its own right: the cascade does not simply produce a sharper picture overall: its benefit is concentrated exactly where the physical drivers are relevant, namely the precipitation forecast. Predicting the wind and temperature first improves where and how hard it rains, not the general appearance of the clouds.

Behaviour Across Horizons

The same pattern holds as the forecast is rolled further out. The cascade keeps its rain-CSI lead over the ablation at every horizon we evaluated, out to T+120, even though training only optimized the short T+30 rollout – evidence that the driver-first structure, not memorization of a fixed lead time, is responsible for the improvement.

Conclusion

The experiment answers the project’s central question as posed: on this data, explicitly predicting the physical drivers before the rain measurably improves rain nowcasting, and does so specifically for precipitation rather than for the image as a whole. Two limitations should be stated plainly. First, the absolute CSI values are low, which reflects the sparsity and internal disagreement of the rain ground truth as much as the model itself. Second, the +52% advantage is measured precisely in that sparse-supervision regime, we cannot assume it would remain as large under denser rain data, where the extra supervision might let the simpler end-to-end model close much of the gap. Whether the driver-first benefit persists under dense supervision is therefore an open question, which we return to as future work.

2.6 Lessons Learned

Looking back, some of our decisions held up well and others we would make differently with hindsight.

- **Validate the data before trusting the model.** The swapped-channel bug degraded results silently for some time before we found it. In hindsight we would have built automatic data-integrity checks and quick visual sanity-checks into the ingestion pipeline from the very beginning, rather than treating the data as correct and debugging the model.
- **Recognize the limits of the rain data early.** Rain is only measured at a handful of sparse IMS stations, and this both limits what the model can learn and caps how high the measured CSI can realistically go. We only appreciated how tight that ceiling was partway through the project. Realizing it at the start would have set the right expectations for the absolute scores and pushed us to look for a denser rain source (such as radar) much sooner.
- **Match the selection metric to the goal.** We initially tracked validation loss, then discovered it did not track rain skill. The lesson is to decide up front what actually defines success (here, CSI) and to select and compare models by that from day one.

- **Make the experiment switchable from the start.** Building the cascade so the driver link could be turned off with a single flag was the best methodological choice we made – it kept every cascade-versus-ablation comparison perfectly controlled. We would design any future comparison the same way.
- **Control cost and scope deliberately.** Iterating locally on the A5000 and renting the H100 only for final runs, together with capping the training rollout, kept the project affordable and fast without compromising the final result. This worked, and we would repeat it.

2.7 Meeting the Project Metrics

We set two kinds of targets for the project: the engineering requirements the system had to satisfy, and the quantitative success metrics defined in Phase A.

Engineering Requirements

The functional and non-functional requirements were met. The pipeline synchronizes the multi-source data onto a common 15-minute, 256×256 grid, normalizes all inputs and targets, splits the data strictly by time to prevent leakage, and computes the meteorological metrics (CSI, HSS, SSIM, RMSE). The model persists checkpoints and supports deterministic seeding for reproducible runs, and it validates against two ground-truth sources (dense satellite for clouds, sparse stations for rain). On the non-functional side, the design keeps a clean separation between the data pipeline and the model, and the trained model meets the efficiency goals: at $\sim 30\text{M}$ parameters it runs a full forecast on an ordinary 8 GB GPU, well within the 15-minute data-refresh interval.

Quantitative Success Metrics

The Phase A success metrics are assessed against the held-out test results in Table 2.

Target (set in Phase A)	Result	Status
Cloud SSIM > 0.7	0.81	Met
Rain skill positive (HSS > 0)	0.23	Met
Improve rain over the end-to-end baseline	+52% CSI	Met
Run inference on a consumer GPU	8 GB, $<$ refresh interval	Met
Rain CSI > 0.5	0.135	Not met (see note)

Table 2: **Project metrics assessed on the held-out test set.**

Every target was met except the absolute rain CSI. That figure (> 0.5) was set in Phase A before we understood how strongly the sparse rain ground truth limits the achievable score: with rain observed only at scattered stations, a forecast is penalized as a full miss whenever a correctly predicted storm falls between gauges, which holds the pointwise CSI well below such a target for any model. The metric that actually reflects the project’s research question – whether the driver-first cascade improves rain forecasting – was met decisively.

2.8 Future Work

Several directions would extend this work, in rough order of expected impact:

- **Denser rain ground truth.** The clearest next step is to replace the sparse station gauges with a dense rain source such as weather radar. This would give the rain head far more to learn from and raise the ceiling on the measurable CSI, allowing the driver-first cascade to be pushed toward much higher absolute skill and its benefit to be confirmed at full spatial resolution rather than only at scattered station points.
- **Longer forecast horizons.** Training was capped at a short T+30 rollout to save time and cost. Training the full rollout out to T+120 and beyond – with the extra compute that requires – would let the model be optimized directly for the longer lead times it is currently only extrapolated to.
- **Adaptive loss weighting.** The balance between the cloud, driver, and rain loss terms is currently fixed by hand. Adjusting these weights automatically during training could remove a fragile piece of manual tuning and improve stability.
- **Toward operational use.** The model could be connected to a near-real-time EUMETSAT feed so it produces rolling, automatically-updating forecasts. We have not attempted this, and a real deployment would need to address live data latency, reliability, and validation on incoming data, so it remains an open direction rather than a finished capability.

References

- [1] Peter Bauer, Alan Thorpe, and Gilbert Brunet. The quiet revolution of numerical weather prediction. *Nature*, 525(7567):47–55, 2015.
- [2] EUMETSAT. Meteosat second generation (msg) seviri level 1.5 image data. <https://data.eumetsat.int>, 2025. Accessed: 2025-12-27 via EUMETSAT Data Store API.
- [3] Levi Harris and Tianlong Chen. A space-time transformer for precipitation forecasting, 2025.
- [4] Israel Meteorological Service. Israel meteorological service (ims) data portal: Automated station network data. <https://ims.gov.il/en/dataGov>, 2025. Accessed: 2026-01-01.
- [5] Gavin D. Madakumbura, Chad W. Thackeray, Jesse Norris, Naomi Goldenson, and Alex Hall. Anthropogenic influence on extreme precipitation over global land areas seen in multiple observational datasets. *Nature Communications*, 12(1):3944, 2021.
- [6] Jimeng Shi, Azam Shirali, Bowen Jin, Sizhe Zhou, Wei Hu, Rahuul Rangaraj, Shaowen Wang, Jiawei Han, Zhaonan Wang, Upmanu Lall, Yanzhao Wu, Leonardo Bobadilla, and Giri Narasimhan. Deep learning and foundation models for weather prediction: A survey, 2025.
- [7] Xingjian Shi, Zhourong Chen, Hao Wang, Dit-Yan Yeung, Wai-Kin Wong, and Wang-chun Woo. Convolutional lstm network: A machine learning approach for precipitation nowcasting. In *Advances in Neural Information Processing Systems*, volume 28, 2015.
- [8] Cheng Tan, Zhangyang Gao, Siyuan Li, and Stan Z. Li. Simvpv2: Towards simple yet powerful spatiotemporal predictive learning. *IEEE Transactions on Multimedia*, 27:5170–5184, 2025.

A User Guide

This guide is for a researcher who intends to continue this work — to reproduce the reported results, inspect the model’s forecasts, and run the driver-first cascade experiment. It covers operating the delivered system, not building or deploying it. For environment setup, retraining, and internals, see the Maintenance Guide (Appendix B). Two use scenarios are covered:

1. **Reproducing results** from the delivered checkpoints — the primary use.
2. **Viewing results through the standalone offline demo**, with no setup.

A.1 What the System Does

Given the last hour of weather-satellite imagery over Israel, the system forecasts the next two hours (in 15-minute steps) of four fields: surface wind, surface temperature, cloud structure, and rain intensity. Its distinguishing feature is the Driver-First Cascade: it predicts the physical drivers (wind, temperature) first and conditions the cloud and rain forecast on them. The system ships with two checkpoints — the Driver-First Cascade and the End-to-End Ablation (the same network with the driver link switched off) — so the core experiment can be reproduced directly.

A.2 Use Scenario 1: Reproducing the Results

This scenario runs the delivered models on the held-out test set and reports the skill scores behind the results in the main text. It requires the environment and data prepared as in Appendix B.

A.2.1 Prerequisites

- **Environment.** The Python environment created as described in Appendix B (Section B.1).
- **Model weights.** The two trained models, `driver_first.pt` and `end_to_end.pt`, are included in the repository under `weights/`.
- **Processed dataset.** The processed data (satellite frames and the test index) is too large to store in git and is not included in the repository. To run the evaluation, rebuild it from the raw sources by following the data pipeline in Appendix B. The pipeline writes the processed files into `data/processed/`, where the evaluation reads them from.

A.2.2 Step 1: Run the Held-Out Evaluation

`eval_all_test.py` evaluates both the Driver-First Cascade and the End-to-End Ablation on the strictly held-out test set and reports the per-horizon skill scores (CSI, HSS, SSIM, POD, FAR) for each:

```
python scripts/eval_all_test.py --sub 10000 --horizons 1,2,3,4,6,8
```

Results are written under `eval_results/test/`. CSI is pooled across all station readings, so clear days count as true negatives rather than inflating the score. The gap between the cascade and the ablation in this output is the central result of the project: because the two models differ only in whether the driver fields reach the second stage, the difference is precisely the contribution of the driver-first ordering.

A.2.3 Interpreting the Outputs

- **CSI** (0–1, higher is better) — the rain hit-skill, the gap between cascade and ablation is the key result. Absolute values are modest because the rain ground truth is sparse.
- **SSIM** (0–1) — how closely the predicted cloud field matches the observed one.
- **RMSE** — the error of the predicted wind and temperature fields.
- A CSI of 0 on a dry case is expected — there is no rain to score, so the cloud SSIM is the relevant number.

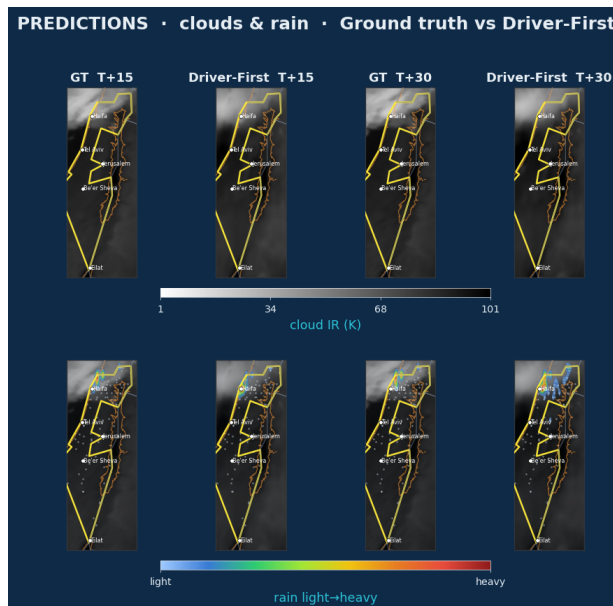


Figure 6: **Example forecast output.** Predicted cloud (top) and rain (bottom) fields from the Driver-First Cascade against the observed ground truth, at T+15 and T+30.

A.3 Use Scenario 2: The Offline Demo

For a no-setup visual overview, the project includes a standalone Windows viewer that plays back a pre-rendered bank of forecast cases with no Python environment or GPU.

1. Copy the folder containing `HorizonForecastDemo.exe` to the machine and double-click it. The first launch self-extracts and takes a few seconds.
2. Use the **Case** dropdown to pick a weather situation, set **Model** to Compare, and scrub the **Horizon** slider to the peak noted in the caption.

3. The Driver-First Cascade catches the rain near the correct stations while the End-to-End ablation misses or under-predicts the CSI values update to show the gap.

The demo does not run the network live. Producing forecasts for new dates uses the tools in Scenario 1.

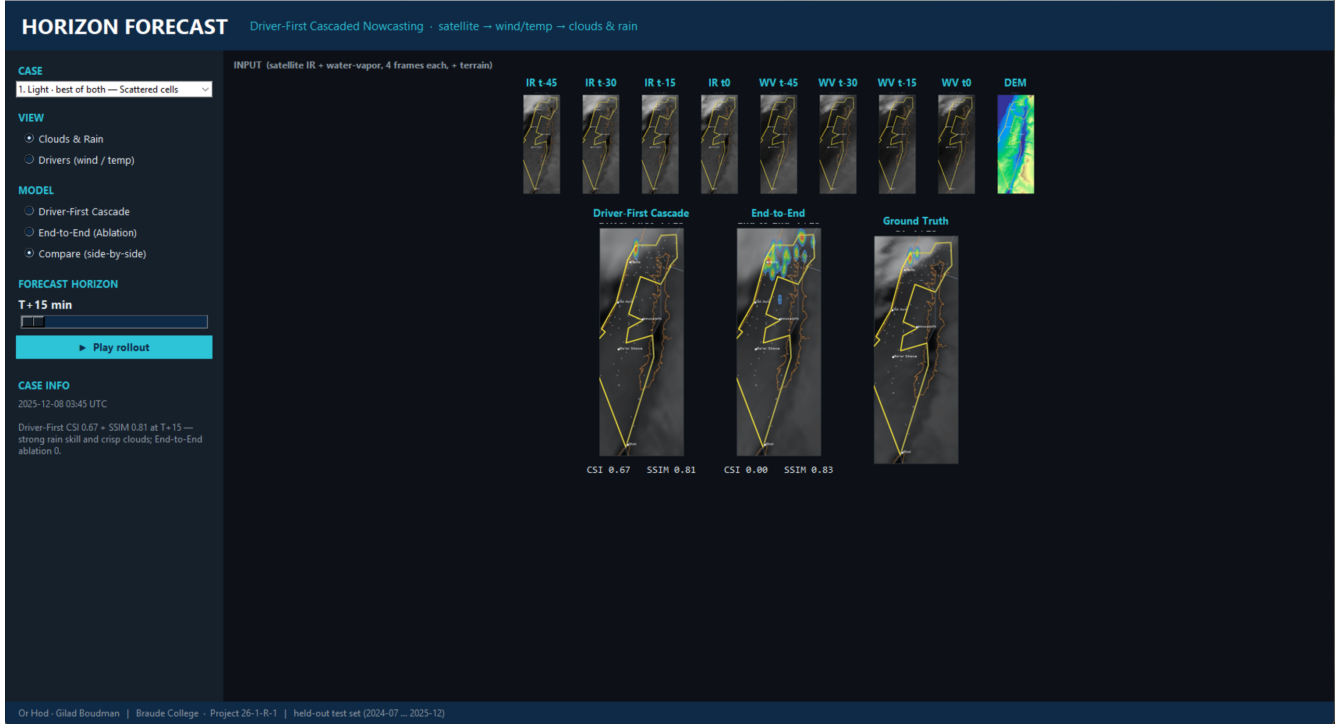


Figure 7: **The offline demo application (Compare mode).** Left: the controls — case selector, View (Clouds & Rain / Drivers), Model (Cascade / Ablation / Compare), the forecast-horizon slider, and Play. Right: the four most recent IR and WV satellite frames plus the terrain map (top row), and the Driver-First Cascade, End-to-End Ablation, and Ground Truth forecasts with their CSI and SSIM (bottom row). On this case at T+15 the cascade reaches CSI 0.67 while the ablation misses the rain entirely (CSI 0.00).

A.4 Scope and Limitations

- The system nowcasts 0–2 hours ahead over Israel only.
- The absolute rain CSI is bounded by the density of the ground truth. The meaningful outcome is the cascade-vs-ablation comparison.
- Reproducing forecasts on new dates requires rebuilding the dataset through the data pipeline (Appendix B).

B Maintenance and Developer Guide

This guide is for a developer who needs to set up, run, debug, retrain, or extend the system. It assumes familiarity with Python, PyTorch, and the command line. End-users should instead see the User Guide (Appendix A).

B.1 Operating Environment

Hardware. An NVIDIA GPU is required for training, at least 24 GB VRAM is recommended (development used an RTX A5000 24 GB, and the final full-dataset runs used a cloud H100). Inference and the demo run comfortably on a consumer card (an RTX 3070 8 GB) or on CPU.

Software. Developed on Windows 11, the training scripts also run on Linux. Python 3.13. Install the CUDA build of PyTorch first (to get the correct wheel for your driver), then the remaining dependencies:

```
python -m venv venv
venv\Scripts\activate
pip install torch --index-url https://download.pytorch.org/whl/cu128 #
    match your CUDA driver
pip install -r requirements.txt
```

`requirements.txt` covers training, evaluation, figure rendering (`matplotlib`), and the demo build (`pyinstaller`). No third-party infrastructure (databases, servers) is required.

B.2 Repository Structure

The project is a single Python package (`src/`) plus a set of top-level entry points and supporting folders. The most important files are described below, grouped by role.

Entry points (run these directly):

`entry_point.py` The training command-line interface. It parses every training flag, builds the data loaders and the loss, and runs the trainer. This is the file the launcher scripts in `scripts/launch/` call.

`run_viz.py` Loads a trained checkpoint, runs inference on a few validation or test samples, and saves one forecast-dashboard image per sample — the quickest way to eyeball a model’s predictions.

The `src` package:

`src/data/dataset.py` Defines `HorizonDataset`, which assembles each training sample: the 12-channel input (four frames of IR, WV, and DEM), the future satellite frames, the sparse IMS rain classes, the dense ERA5 wind/temperature, and the station mask. It also holds the 64 rain-intensity bucket definitions and the default IR/WV channel correction.

`src/data/prep.py` The ten-stage preprocessing pipeline that turns the raw downloads (satellite TIFFs, IMS CSVs, ERA5, SRTM) into the normalized `.npy` frames, the station mask, and the chronological train/validation/test index CSVs that the dataset reads.

src/models/model.py The full network — the SimVPv2 encoder, the Stage-1 PhysicsDriver-Head, and the Stage-2 ManifestationHead. The ablation switch lives here: `HorizonForecastModel(no_cascade=True)` zeroes the driver fields before Stage 2.

src/train/train.py The training machinery: `HorizonLoss` (the multi-objective loss), `Trainer` (the two-phase frozen schedule, scheduled-sampling rollout, and checkpointing), and `load_model_for_inference` (used by every evaluation and visualization entry point).

src/eval/evaluate.py The metric implementations (CSI, HSS, POD, FAR, SSIM) and the multi-horizon evaluation loop that rolls a model out and scores it at each lead time.

src/eval/inference.py The autoregressive rollout helper that feeds a model’s own output back as its next input to produce a multi-step forecast.

src/eval/baselines.py Reference baselines (such as persistence) used to sanity-check that the model beats trivial predictors.

src/viz/visualize.py The plotting helpers that render the forecast dashboards, shared by `run_viz.py` and by the periodic visualizations during training.

Supporting folders:

scripts/ Standalone tools: `eval_all_test.py` (held-out test evaluation of the delivered models), `export_weights.py` (exports the two checkpoints as the committed FP16 weights), `render_longhorizon.py` (renders the qualitative forecast figures — cascade vs. ablation vs. ground truth — and provides the rendering machinery the demo builder reuses), `render_demo_assets.py` (builds the demo’s pre-rendered case bank), `build_test_index.py` (rebuilds the held-out test index), and `launch/` (the two-phase training launchers).

data/ The data-acquisition scripts (`download_seviri.py`, `download_ims.py`, `download_ims_stations.py`, `download_era5.py`, `download_srtm.py`), the ERA5 processing script, map borders, and the four small committed run-artifacts needed to load the model.

demo/ The standalone offline viewer: `app.py` (the Tkinter application) and `build_demo_exe.bat` (packages it into `HorizonForecastDemo.exe`).

weights/ The two delivered models as half-precision checkpoints (`driver_first.pt`, `end_to_end.pt`).

experiments/ A self-contained supporting analysis (the neighborhood-CSI tolerance study).

Large artifacts — `checkpoints/`, the heavy `data/` subdirectories, and `logs/` — are git-ignored.

B.3 Dataset Acquisition

The training data is not stored in git (the raw sources and processed frames total hundreds of GB). Only four small artifacts needed to *load* a trained model are committed: `dem_256.npy`, `norm_stats.json`, `station_mask.pt`, and `rain_weights.pt`. To rebuild the full dataset, first download the four raw sources below — each needs its own free account or API key — then run the preprocessing pipeline.

1. Satellite — EUMETSAT SEVIRI (model input)

IR $10.8\mu\text{m}$ and WV $6.2\mu\text{m}$ tiles cropped to the Israel domain, fetched by `data/download_seviri.py` through the EUMETSAT Data Store and Data Tailor (the `eumdac` client). Create a free EUMETSAT account, generate a consumer key and secret at <https://api.eumetsat.int/api-key/>, and in the Data Tailor create a preset that crops to the region of interest and outputs GeoTIFF. Then:

```
set EUMETSAT_KEY=your_consumer_key
set EUMETSAT_SECRET=your_consumer_secret
python data/download_seviri.py
```

Output goes to `data/raw/IR_108` with `WV_062 Tif (EUMETVIEW) /`.

2. Reanalysis — ERA5 (driver ground truth)

The 10 m wind components and 2 m temperature from the Copernicus Climate Data Store. Register at <https://cds.climate.copernicus.eu/>, then create a `.cdsapirc` file holding your key:

```
url: https://cds.climate.copernicus.eu/api
key: YOUR_API_KEY
```

Then download and regrid it:

```
pip install cdsapi netCDF4 xarray
python data/download_era5.py      # NetCDF -> data/era5/
python data/process_era5.py      # regridded -> data/era5_npy/
```

3. Terrain — NASA SRTM (static input)

One-arc-second elevation, pulled from a public AWS mirror (no account needed):

```
python data/download_srtm.py
```

This source is already provided pre-processed as the committed `dem_256.npy`; it only needs re-fetching for a full from-scratch rebuild.

4. Ground stations — IMS (sparse rain ground truth)

Per-station wind, temperature, and precipitation for roughly 100 stations, fetched from the Israel Meteorological Service API. Request a free API token at <https://ims.gov.il/en/ObservationDataAPI>, then:

```
set IMS_API_TOKEN=your_token
python data/download_ims_stations.py  # station coordinates
python data/download_ims.py           # per-station time series
```

Rebuilding the processed dataset

With the four raw sources in `data/raw/`, run the full preprocessing pipeline — all ten stages, in order — with a single command from the repository root:

```
python -m src.data.prep
```

The stages clean and geolocate the station records, build the elevation map, convert the satellite tiles, merge and snapshot the IMS data, build the station mask, construct the chronological train/validation/test indices, and compute the normalization statistics and rain-class weights. Each stage skips work already on disk, so the multi-day rebuild is resumable. (The held-out test index alone can be rebuilt with `python scripts/build_test_index.py`.)

B.4 Data Pipeline

The preprocessing (`src/data/prep.py`) converts the raw sources into the tensors the model consumes: satellite TIFFs become normalized 256×256 .`numpy` frames, IMS GPS coordinates become the pixel station mask, ERA5 is regridded to the same grid, and per-forecast-moment index CSVs are built for the train/val/test splits. At training time `HorizonDataset` assembles, per sample: the 12-channel input, the future satellite frames, the IMS rain classes at stations, the dense ERA5 wind/temp, and the station mask.

Grid: lat 29–34°N, lon 34–36°E, 256×256 . Splits: train 2020–2023, validation 2024 H1, test 2024-07 → 2025-12.

IR/WV channel order. The raw EUMETVIEW TIFs have the IR and WV bands swapped on alternate frames. `HorizonDataset` corrects this automatically by default (normalizing IR to channel 0), matching the layout the delivered checkpoints were trained on, so no configuration is needed. Only to reproduce the original, pre-fix models should you set `HORIZON_FIX_CHANNELS=0` to disable the correction.

B.5 Training

The method is a two-phase frozen schedule — first train the encoder and driver head with the manifestation head frozen, then freeze the encoder and driver head and train the manifestation head on top:

```
# Phase 1 -- encoder + drivers (rain head frozen), dense ERA5 thermo
python entry_point.py --era5-path data\era5_numpy --freeze-stage mani ^
    --lambda-cloud 0 --lambda-rain 0 --max-epochs 30 --ckpt-dir checkpoints\
    h5_phase1

# Phase 2 -- freeze encoder+drivers, train rain/cloud head (resume Phase-1
    weights)
python entry_point.py --era5-path data\era5_numpy ^
    --resume-ckpt checkpoints\h5_phase1\gpu0_best.pt --resume-weights-only ^
    --freeze-stage encoder_phys --lambda-thermo 0 --lambda-cloud 1.0 --
    lambda-rain 0.5 ^
    --max-epochs 30 --ckpt-dir checkpoints\h5_phase2
```

Flag	Meaning
<code>--freeze-stage</code>	Which part of the network to freeze. Phase 1 uses <code>mani</code> (freeze the Manifestation Head — train encoder + driver head only); Phase 2 uses <code>encoder_phys</code> (freeze the encoder + driver head — train the Manifestation Head only).
<code>--era5-path DIR</code>	Dense ERA5 wind/temp supervision.
<code>--no-cascade</code>	Ablation — zero the drivers fed to Stage 2.
<code>--lambda-cloud,</code> <code>rain, -thermo</code>	- Loss-term weights.
<code>--resume-ckpt,</code> <code>resume-weights-only</code>	-- Load weights only (reset epoch/LR); used between phases.

Selection rule. Select the checkpoint by rain CSI, not validation loss. In Phase 2 the validation loss can rise while CSI keeps climbing. Use the multi-horizon evaluation to pick the best-CSI epoch. Each 30-epoch phase takes roughly 8–10 hours on an H100. A full cascade (Phase 1 + Phase 2) is therefore about 16–20 hours.

Reusing Phase 1. Because Phase 1 freezes the rain head, any Phase-2 variant (including the ablation) can resume the same Phase-1 checkpoint — do not re-run Phase 1 for a rain-head-only experiment.

B.6 Evaluation

```
python scripts\eval_all_test.py --sub 10000 --horizons 1,2,3,4,6,8
```

By default this evaluates the two delivered models (the Driver-First Cascade and the End-to-End Ablation). Set `HORIZON_EVAL_MODELS="Name:path,..."` to evaluate other checkpoints. CSI is pooled (every station reading is concatenated across all samples, then one CSI is computed), so clear days count as true negatives. A self-contained analyses live under `experiments/:neighborhood_csi/` (tolerance-CSI curve).

B.7 Extending the System

To add a new experiment or hypothesis:

1. **Loss/target change:** edit `HorizonLoss.forward` and/or `HorizonDataset`, adding a dataset flag and a loss branch (opt-in, so the defaults are unchanged).
2. **CLI flag:** add it in `entry_point.py` and thread it through to the data loaders and the loss.
3. **Smoke-test** the new path on a few samples before launching a full run.
4. **Launcher:** copy a `scripts/launch/launch_*.bat`, change the checkpoint directory, log, and the new flag.

B.8 Rebuilding the Demo

```
python scripts\render_demo_assets.py    # rebuild demo/assets + manifest.json
    (both delivered models)
demo\build_demo_exe.bat                  # -> demo/dist/HorizonForecastDemo.
exe
```

To change which cases appear, edit the CASES list in `render_demo_assets.py`. Close any running demo window before rebuilding — the running executable locks `dist/`.

B.9 Troubleshooting

Symptom	Cause / fix
Scrambled inputs / black predictions	<code>HORIZON_FIX_CHANNELS=0</code> is set, disabling the default band-order correction. Unset it — the correction is on by default.
CUDA out of memory	Lower <code>--batch-size</code> and <code>--rollout-max</code> . Ensure gradient checkpointing is on (<code>USE_GRAD_CKPT=True</code> in <code>model.py</code>). Reduce <code>--num-workers</code> on a memory-capped container.
Cross-entropy assertion on class index	A rain class fell outside <code>[0, 63]</code> . Clamp class indices to <code>[0, N_RAIN_BINS-1]</code> .
Validation loss rises but CSI improves	Expected in Phase 2 — select by CSI, not loss.
Demo build fails (“dist busy”)	A demo window is open, close it before rebuilding.

B.10 Conventions

- BF16 autocast, no gradient scaler (bf16 has the range), gradient checkpointing for rollout memory.
- GroupNorm rather than BatchNorm — stable at batch size 1 for inference.
- Bilinear upsampling + convolution rather than transposed convolution — avoids checkerboard artifacts.
- 64 log-spaced rain buckets with a class-weighted cross-entropy to counter the $\sim 98.8\%$ dry dominance.